

GitHub Repo Link:

[Aadeleye11/Project-1_Assignment_Formative](#)

Question 1: C Program Development and Compilation for a Sensor Monitoring System

Complete C Source Code

```
// Function to calculate the Water Quality Index
float calculateIndex(float temp, float turbidity) {
    float tempDeviation = fabs(temp - 25.0);
    float turbidityPenalty = turbidity / 2.0;
    return 100.0 - (tempDeviation + turbidityPenalty);
}

// Function to classify and print the formatted report
void printReport(float temp, float turbidity, float index) {
    printf("\n==== WATER QUALITY REPORT =====\n");
    printf("Temperature: %.2f C\n", temp);
    printf("Turbidity:   %.2f NTU\n", turbidity);
    printf("Index Score: %.2f\n", index);
    printf("Status:      ");

    if (index >= 80) {
        printf("Good\n");
    } else if (index >= 60) {
        printf("Warning\n");
    } else {
        printf("Critical\n");
    }
}
```

```

        printf("=====\n");
    }

int main(void) {
    float temperature, turbidity;

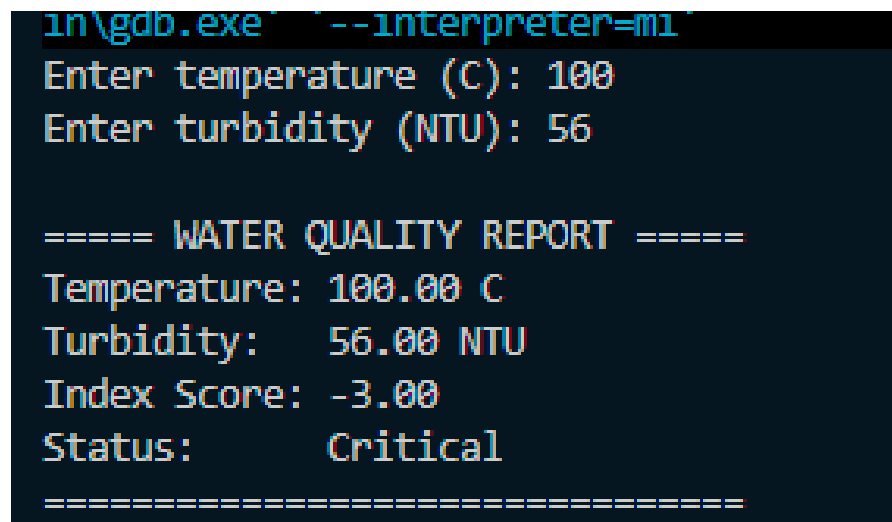
    printf("Enter temperature (C): ");
    scanf("%f", &temperature);
    printf("Enter turbidity (NTU): ");
    scanf("%f", &turbidity);

    float index = calculateIndex(temperature, turbidity);
    printReport(temperature, turbidity, index);

    return 0;
}

```

Sample Output from one test run



```

in\gdb.exe --interpreter=mi
Enter temperature (C): 100
Enter turbidity (NTU): 56

==== WATER QUALITY REPORT ====
Temperature: 100.00 C
Turbidity: 56.00 NTU
Index Score: -3.00
Status: Critical
=====

```

Short Technical Explanation covering a, b, and c

a. Real World-Applications: Embedded systems would use c. For example, IoT environmental monitoring devices, or the microcontrollers in Engine Control Units (ECUs) in cars. C would be great for such systems as it can produce highly efficient machine code, allows access to lower levels of memory to manipulate hardware registers and uses very little memory.

b. Errors:

- Syntax error - forgetting a semi colon at the end of a line (for example float index = 100). This is a syntax error as it breaks the grammar of the C language and therefore the compiler will not be able to create the program from this code.
- Semantic error - using $100 + (\text{tempDeviation} + \text{turbidityPenalty})$ rather than subtraction. This code will compile without issues but will produce erroneous results at runtime due to incorrect logic within the code. This is a semantic error.

c. Compilation lifecycle:

- Preprocessing: Processes directives like `#include`. (Input: `.c` source code -> Output: `.i` expanded source code).
- Compilation: Translates the code into assembly language. (Input: `.i` file -> Output: `.s` assembly code).
- Assembly: Converts assembly instructions into machine-level binary object code. (Input: `.s` file -> Output: `.o` object file).

- Linking: Combines object files and standard libraries (like `math.h`) into the final runnable program. (Input: `.o` files -> Output: `.exe` or `a.out` executable).

Question 2: Transaction Processing and Control Flow

Complete C Source Code

```
int main() {
    int choice;
    float balance = 0.0;
    float amount;
    int depositCount = 0;
    int withdrawCount = 0;

    while (1) {
        printf("\n===== MOBILE MONEY TRANSACTION SYSTEM =====\n");
        printf("1. Deposit\n2. Withdraw\n3. Check Balance\n4. Transaction\n5. Exit\n");
        printf("Enter choice: ");

        // Handle non-integer input
        if (scanf("%d", &choice) != 1) {
            while(getchar() != '\n');
            printf("Invalid input. Please enter a number.\n");
            continue;
        }

        if (choice == 5) {
```

```

        printf("System terminated.\n");
        break; // Exits the while loop entirely
    }

    switch (choice) {
        case 1:
            printf("Enter deposit amount: ");
            scanf("%f", &amount);
            if (amount <= 0) {
                printf("Transaction rejected: Amount must be
positive.\n");
                continue; // Skips the rest of the loop and returns to
menu
            }
            balance += amount;
            depositCount++;
            printf("Deposit successful.\nCurrent balance: %.2f RWF\n",
balance);
            break; // Exits the switch statement

        case 2:
            printf("Enter withdrawal amount: ");
            scanf("%f", &amount);
            if (amount <= 0) {
                printf("Transaction rejected: Amount must be
positive.\n");
                continue; // Skips to menu
            }
            if (amount > balance) {
                printf("Transaction rejected: Insufficient balance.\n");
                continue; // Skips to menu
            }
            balance -= amount;

```

```

        withdrawCount++;
        printf("Withdrawal successful.\nCurrent balance: %.2f RWF\n",
balance);

        break;

    case 3:
        printf("Current balance: %.2f RWF\n", balance);
        break;

    case 4:
        printf("--- Transaction Summary ---\n");
        printf("Successful Deposits: %d\n", depositCount);
        printf("Successful Withdrawals: %d\n", withdrawCount);
        break;

    default:
        printf("Invalid choice. Please select an option from 1 to
5.\n");
    }
}

return 0;
}

```

Sample Output from one test run

```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 1
Enter deposit amount: 700000
Deposit successful.
Current balance: 700000.00 RWF

```

```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 2
Enter withdrawal amount: 800000
Transaction rejected: Insufficient balance.

```

```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 1

```

```

===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 2

```

```
===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 3
Current balance: 600000.00 RWF
```

```
===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 4
--- Transaction Summary ---
Successful Deposits: 1
Successful Withdrawals: 1
```

```
===== MOBILE MONEY TRANSACTION SYSTEM =====
1. Deposit
2. Withdraw
3. Check Balance
4. Transaction Summary
5. Exit
Enter choice: 5
System terminated.
```

Brief explanation of how the program uses conditionals, loops, break, and continue.

- a. **Conditionals:** Switch/if statements handle the flow through the menu. Within the switch cases, if statements are used for business logic (make sure money is available and all amounts are positive).
- b. **Loops (while (1)):** An infinite while loop wraps the entire program. This allows the agent to always see the menu and does not require the program to be restarted after each transaction.
- c. **Break :** break is used in two ways. Within the switch it is used to prevent one case continuing onto the next case. For option 5, a break is used to exit the while(1) loop which causes the program to end.
- d. **Continue:** continue is used right after a data entry mistake occurs such as entering a negative deposit amount. This causes the rest of the arithmetic and print statements for that transaction to be skipped and the menu loop to begin again.

Question 3: Functions & Recursive Problem Solving

Complete C Source Code

```
// Calculate total distance iteratively

int calcTotal(int arr[], int n) {

    int sum = 0;

    for (int i = 0; i < n; i++) sum += arr[i];

    return sum;

}


// Calculate average showing function reuse by calling calcTotal

float calcAverage(int arr[], int n) {

    if (n == 0) return 0;

    int total = calcTotal(arr, n);

    return (float)total / n;

}


// Find the longest route

int findLongest(int arr[], int n) {

    int max = arr[0];

    for (int i = 1; i < n; i++) {

        if (arr[i] > max) max = arr[i];

    }

}
```

```

        return max;
    }

    // Count routes above a limit
    int countAboveLimit(int arr[], int n, int limit) {
        int count = 0;
        for (int i = 0; i < n; i++) {
            if (arr[i] > limit) count++;
        }
        return count;
    }

    // Recursive function to sum distances
    int recursiveSum(int arr[], int n) {
        // If no elements are left, the sum is 0.
        if (n <= 0) {
            return 0;
        }
        // Add the last element to the sum of the remaining array
        return arr[n - 1] + recursiveSum(arr, n - 1);
    }

    int main() {
        int n;
        int limit;

        // Ask the user for the number of routes

```

```
printf("Enter the number of delivery routes: ");

scanf("%d", &n);

if (n <= 0) {

    printf("Invalid number of routes. Exiting program.\n");

    return 1;

}

// Declare an array of size N

int distances[n];

// Loop to ask the user to input each distance

for (int i = 0; i < n; i++) {

    printf("Enter distance for route %d (in km): ", i + 1);

    scanf("%d", &distances[i]);

}

// Ask for the distance limit

printf("Enter the distance limit for analysis (in km): ");

scanf("%d", &limit);

// Display the results

printf("\n===== DELIVERY DISTANCE ANALYSIS =====\n");

printf("Total distance: %d km\n\n", calcTotal(distances, n));

printf("Average distance: %.2f km\n", calcAverage(distances, n));

printf("Longest route: %d km\n", findLongest(distances, n));
```

```
    printf("Routes above %d km: %d\n\n", limit, countAboveLimit(distances, n,
limit));

    printf("Recursive sum: %d km\n", recursiveSum(distances, n));

    return 0;
}
```

Sample Output from one test run

```
Enter the number of delivery routes: 6
Enter distance for route 1 (in km): 34
Enter distance for route 2 (in km): 56
Enter distance for route 3 (in km): 87
Enter distance for route 4 (in km): 100
Enter distance for route 5 (in km): 345
Enter distance for route 6 (in km): 67
Enter the distance limit for analysis (in km): 60

===== DELIVERY DISTANCE ANALYSIS =====
Total distance: 689 km

Average distance: 114.83 km
Longest route: 345 km
Routes above 60 km: 4

Recursive sum: 689 km
```

Brief explanation of how the program is divided into functions

As you can see from the code, the program adheres to the Single Responsibility Principle. All of the business is in separate pieces (calcTotal, findLongest, etc.). This allows for easier testing and reuse (calcAverage uses calcTotal instead than repeating the code that sums up the array), and leaves main() relatively simple.

Brief explanation of how the recursive function works, including its base case

recursiveSum uses an array and the size of the array (n). The base case uses if (n <= 0) return 0; This implies that if the array is empty then the sum of that array is zero.

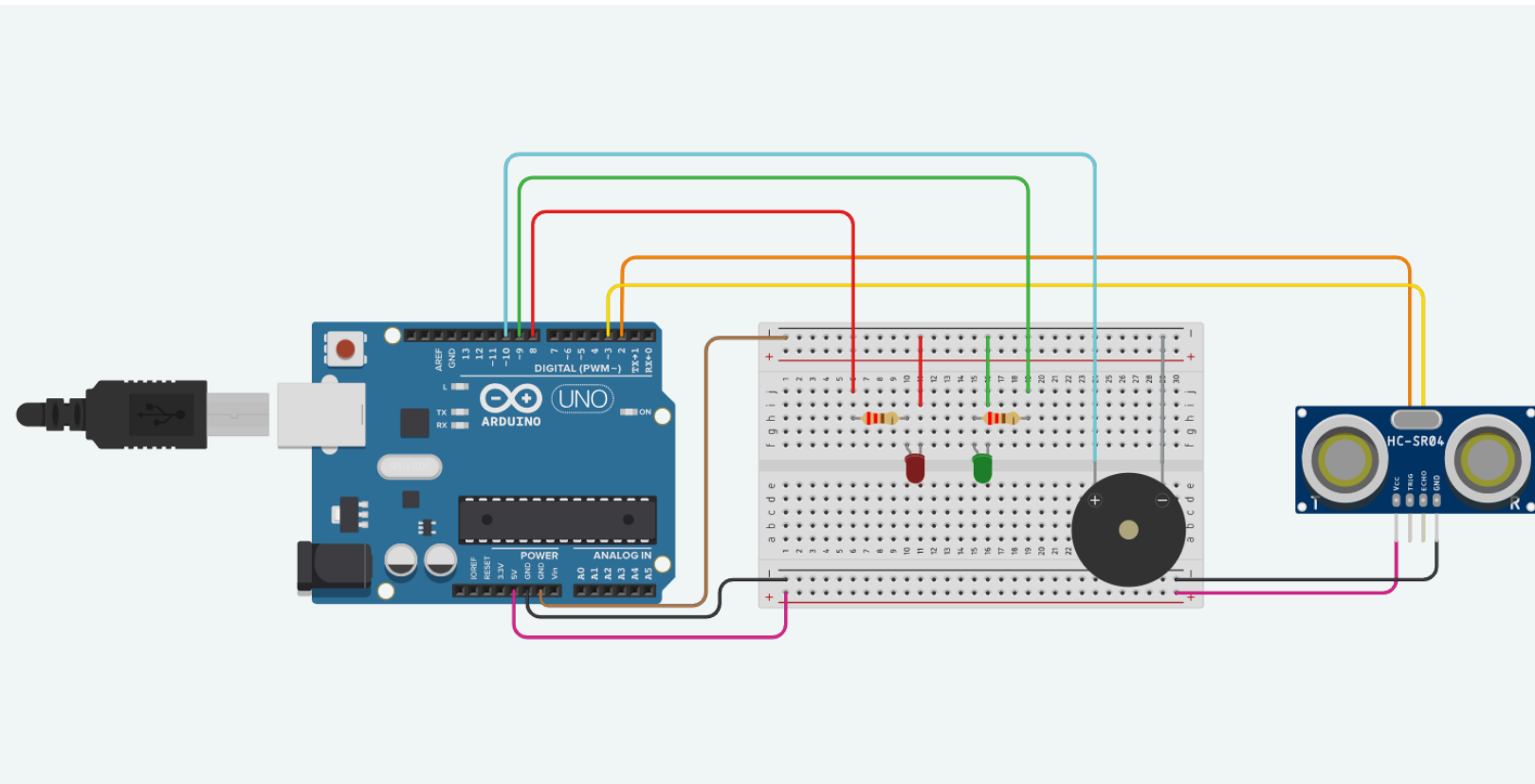
Otherwise, if $n > 0$ it will return `arr[n-1]` plus the recursive function with parameter $n-1$. It successfully reduces the problem to the base case.

Pros and Cons of using recursion for this problem

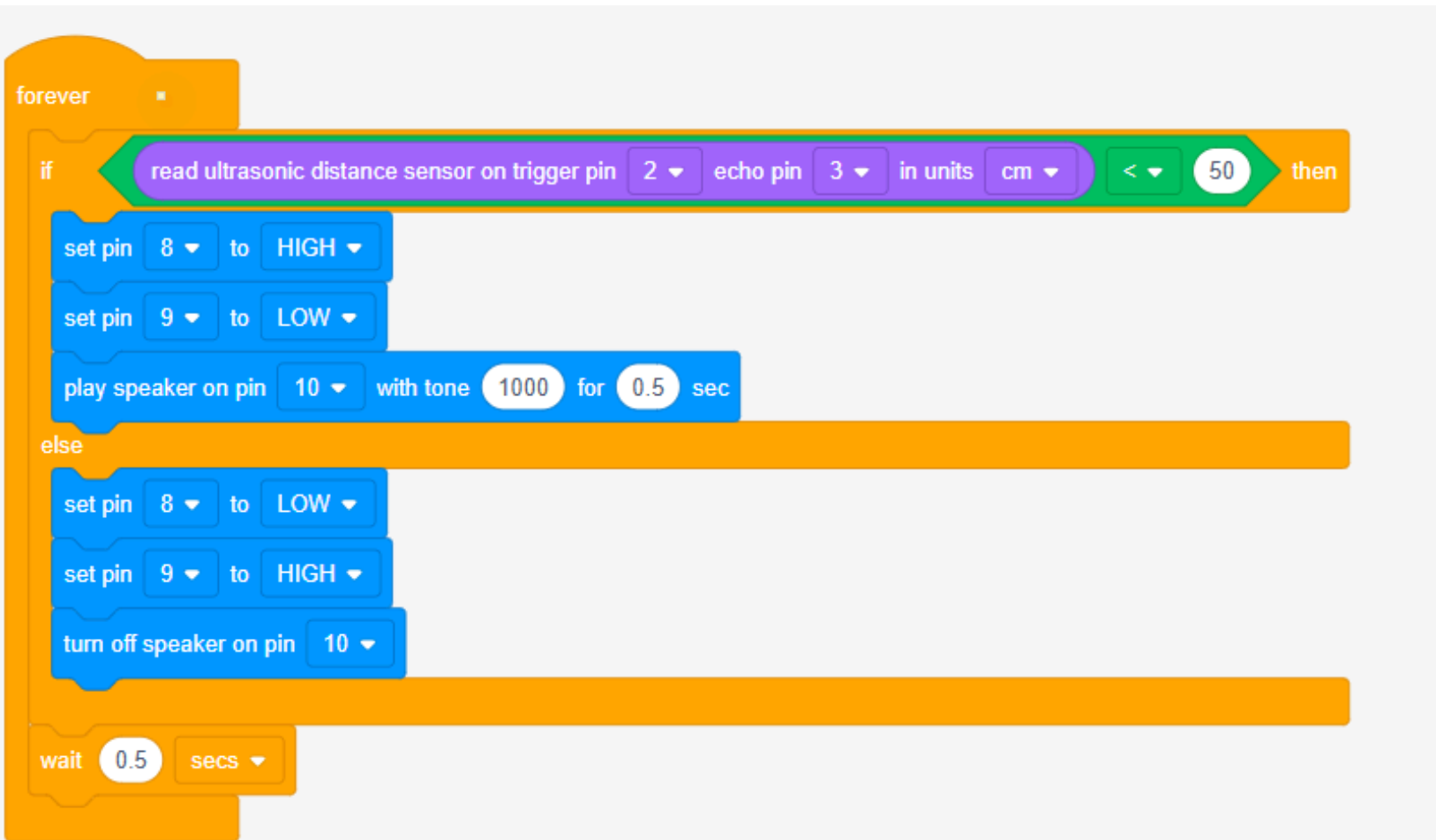
- **Pros of recursion:** It can provide a very clear and readable solution to problems that naturally break down into smaller instances of themselves (e.g. tree or directory data structures, file searching).
- **Cons of recursion:** It uses more memory for storing the call stack. With large arrays it can lead to a stack overflow whereas iteration with a for loop uses a constant amount of memory.

Question 4: Arduino-Based Smart Parking System

Tinkercad circuit design showing all components and connections.



Block diagram showing the flow of data through the system



Arduino source code used in the Tinkercad simulation

```
// C++ code
//
long readUltrasonicDistance(int triggerPin, int echoPin)
{
    pinMode(triggerPin, OUTPUT); // Clear the trigger
    digitalWrite(triggerPin, LOW);
    delayMicroseconds(2);
    // Sets the trigger pin to HIGH state for 10 microseconds
```

```

    digitalWrite(triggerPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(triggerPin, LOW);
    pinMode(echoPin, INPUT);

    // Reads the echo pin, and returns the sound wave travel time in
microseconds
    return pulseIn(echoPin, HIGH);
}

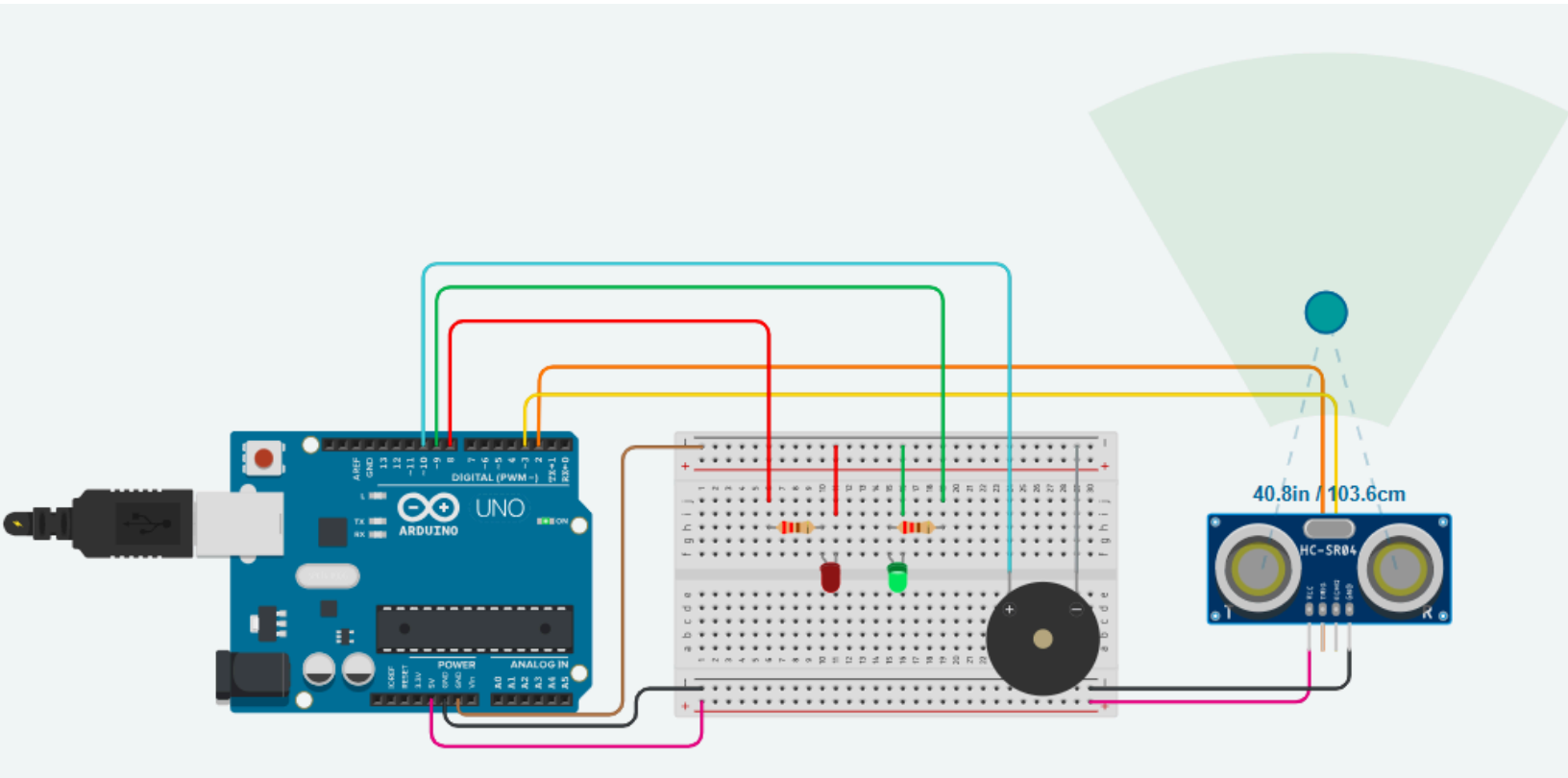
void setup()
{
    pinMode(8, OUTPUT);
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
}

void loop()
{
    if (0.01723 * readUltrasonicDistance(2, 3) < 50) {
        digitalWrite(8, HIGH);
        digitalWrite(9, LOW);
        tone(10, 1.992476322042289e+26, 500); // play tone 1000 (E83 =
1.992476322042289e+26 Hz)
    } else {
        digitalWrite(8, LOW);
        digitalWrite(9, HIGH);
        noTone(10);
    }
    delay(500); // Wait for 500 millisecond(s)
}

```

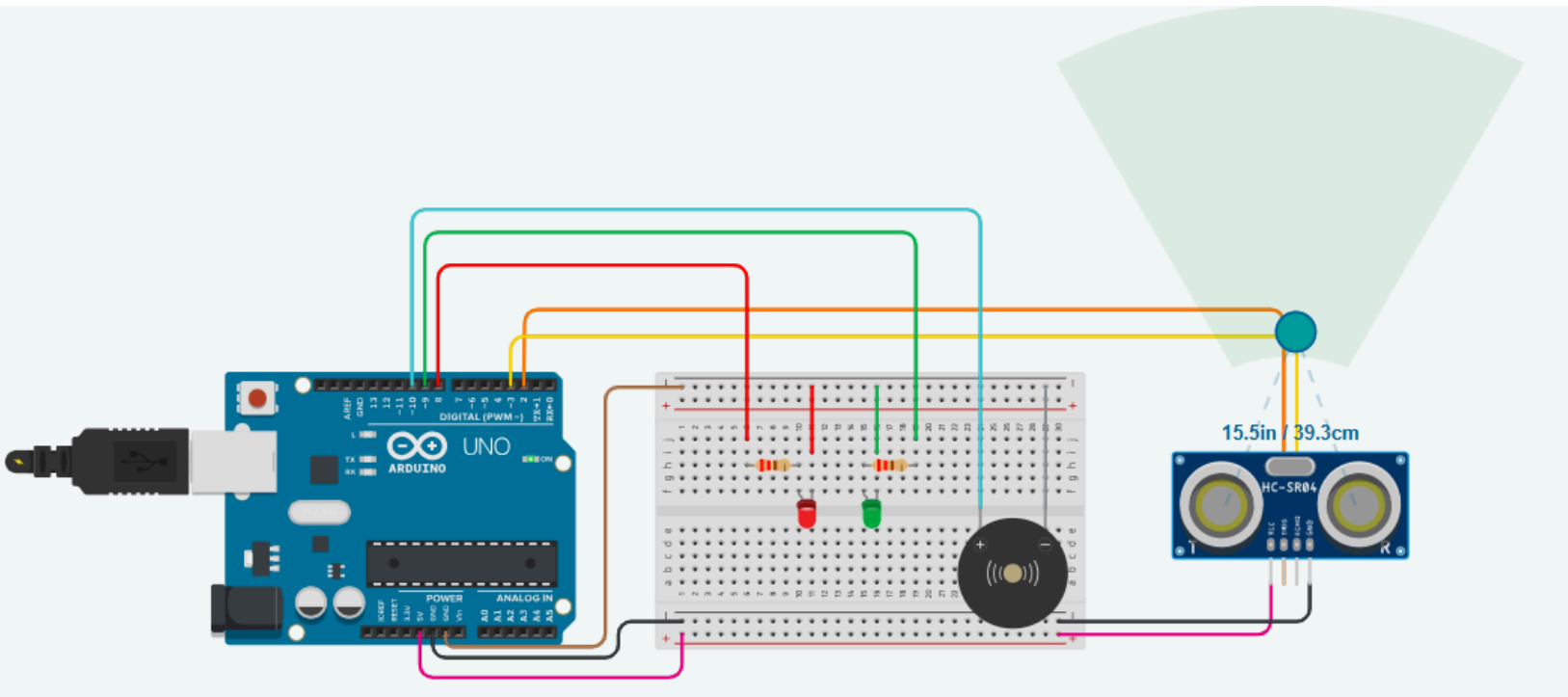
Two simulation test cases

- a. **Vehicle outside the threshold** → Green LED ON, Red LED OFF, Buzzer OFF.



Test Case 1 (Vehicle at 103.6 cm): In Tinkercad, moving the object outside the 50 cm threshold results in the Green LED turning ON, Red LED turning OFF, and the Buzzer remaining silent. The space reads as available.

b. **Vehicle within the threshold** → Red LED ON, Green LED OFF, Buzzer ON.



Test Case 2 (Vehicle at 39.3 cm): Moving the object inside the 50 cm threshold triggers the occupancy logic. The Red LED turns ON, the Green LED turns OFF, and the Buzzer sounds continuously as shown by the buzzing animation shown on the Piezel Buzzer.

Short explanation describing various roles

The ultrasonic sensor takes input from the surrounding environment. Arduino Uno is the controller which converts timing into distance and performs computations. Buzzer and LED's are output devices which convert the data from digital from of Arduino into audio and visual indication to the driver.